2D or not 2D?

-bholechature

Introduction

Every 3D model begins with an idea, and sometimes that idea is nothing more than a few lines on a piece of paper. Yet, translating those lines into a usable three-dimensional object can require hours of modelling expertise, specialised software, and considerable manual effort. During the 24-hour TechZephyr 4.0 hackathon at IIT Bhubaneswar, Team TechZephyr set out to explore a simpler possibility: what if a rough sketch could become the starting point for a 3D model, rather than the beginning of hours of manual work?

This idea led to 2DorNot2D AI (2DorNot2D), an AI-powered application that converts hand-drawn sketches into downloadable 3D assets. Users can draw or upload a sketch, optionally describe the intended object through a text prompt, and generate a model in .glb or .obj format. Our objective was not simply to generate an image that looked three-dimensional, but to build a functional pipeline that produces genuine geometry, remains reproducible across supported computing environments, and can be used beyond the demonstration itself.

Working within a 24-hour deadline required us to balance ambition with feasibility. We prioritised an end-to-end system, measurable geometric fidelity, hardware flexibility, and transparent documentation over relying on a single high-quality output from a specialised model. The result is a working foundation for making 3D creation more accessible to people who can imagine and sketch an object but may not have the time or expertise to model it manually.

The Team and Our Approach

The project was developed through complementary contributions from three team members. Swayam worked on the core pipeline, API, and GPU execution; Disha developed the interactive dashboard, prepared test sketches, and maintained project documentation; and Udit implemented the sketch-processing and generation stages, the candidate-scoring mechanism, and the Blender add-on.

Rather than treating these as isolated components, we structured the system around clearly defined inputs, outputs, and API contracts. This allowed the team to develop different parts in parallel while maintaining compatibility between them. Shared development practices, synchronisation checks, and pull-request reviews helped us integrate the components across Linux, macOS, and Windows environments.

One of our most important decisions was to make the system reproducible rather than optimise exclusively for the hardware available during development. We selected models and designed execution paths with the expectation that the project might be evaluated on a machine without a dedicated NVIDIA

GPU. This constraint influenced our architecture, model selection, memory optimisations, and approach to testing.

Our Solution & Key Features

2DorNot2D AI uses a two-stage generative process. First, Stable Diffusion 1.5 with ControlNet Scribble transforms the input sketch into a refined product-style image while preserving its overall structure. TripoSR then reconstructs this image into a 3D mesh. Following mesh cleanup and validation, the system exports the result for use in compatible 3D applications.

A key feature is our best-of-three candidate selection strategy. Instead of accepting the first generated result, the pipeline creates three candidates and compares their 3D silhouettes against the original sketch using Intersection over Union (IoU). It retains the candidate with the highest score, making geometric correspondence an explicit part of the generation process rather than relying on visual appeal alone.

The application includes an interactive browser dashboard with a live 3D preview, drawing tools, progress indicators, and locally stored design history. It also provides a command-line interface and a Blender add-on that communicates with the local server. Users can export genuine 3D geometry as .glb or .obj files, with vertex colours supported in the generated meshes.

The broader principle behind these features is straightforward: a useful AI tool should do more than produce an impressive preview. Its output should be inspectable, exportable, reproducible, and practical to work with.

Experimental Results and What We Learned

We evaluated the pipeline using 12 pen sketches photographed on paper, each paired with a text prompt. The principal experiments were conducted on a rented NVIDIA RTX PRO 4500 GPU with 32 GB of VRAM.

Selection strategy	Mean silhouette IoU	Lowest IoU	Warm generation time
Single seed-	0.880	0.683	5 seconds

Best of three using 2D cutout-

0.893
0.701

12 seconds

Best of three using 3D mesh-

0.911
0.701

Approximately 16 seconds (estimated)

All 12 sketches produced meshes. The highest scores were recorded for a donut (0.970), a bottle (0.961), and an ice cream (0.951). On a separate set of 18 synthetic cases, selection using 2D cutouts increased mean IoU from 0.809 to 0.851. In one notable case, a chair sketch paired with an incorrect prompt, ‘a horse’, improved from an IoU of 0.384 to 0.937 through candidate selection.

These results reinforced an important engineering lesson: generating multiple outputs is not sufficient unless the system has a meaningful way to choose among them. Our experiments also showed that improvements in one representation do not always translate into improvements in another. A candidate that looks suitable as a 2D cutout can still produce a less accurate 3D mesh, which motivated our final selection strategy.

Performance nevertheless varies across objects. The bulb sketch, for example, achieved an IoU of 0.701 because stray light rays interfered with mask generation. The cloud model appeared flat from the side, while small structural details, such as the number of holes in an object, were not always preserved. These cases helped establish where the current pipeline performs well and where further work is necessary.

Challenges, Decisions, and Engineering Trade-offs

Several challenges required us to revisit implementation decisions rather than simply adjust model parameters. Checkpoint incompatibilities, incorrectly oriented mesh triangles, and coordinate-frame inconsistencies initially prevented reliable reconstruction and export. We addressed these through checkpoint-key remapping, geometry validation using a sphere test, and explicit correction of mesh orientation.

Memory limitations proved particularly significant. TripoSR's default density queries required too much memory for a 4 GB GPU. By introducing chunked density sampling, we reduced the measured requirement to approximately 2.4 GB, allowing the reconstruction stage to run within that constraint.

However, the image-generation stage still exceeds the tested 4 GB configuration, demonstrating that support for low-memory hardware remains incomplete.

We also learned that optimising a complex generative pipeline is rarely a matter of finding one universally superior setting. Adjustments to prompt wording, ControlNet strength, background colour, and foreground ratio sometimes improved individual examples while degrading others. Rather than presenting isolated successes as general improvements, we retained stable defaults and documented unsuccessful experiments.

These decisions reflect an important aspect of the project: the aim was not to conceal the limitations of the underlying models, but to build a system that could be evaluated honestly, with its strengths and weaknesses visible.

Future Scope

The next phase of development will focus on improving the quality and usability of the generated assets. Multi-view reconstruction approaches such as Zero123++ or InstantMesh could help address the limitations of single-view reconstruction, particularly the guessed backs and flattened side profiles of certain objects. Reliable UV texture baking would improve compatibility with conventional 3D workflows, while further memory optimisation could extend image-generation support to smaller GPUs.

Other priorities include removing stray sketch marks before mask generation, supporting multiple objects in a single drawing, and validating the Blender add-on through interactive testing inside Blender. A recorded demonstration and more extensive cross-hardware evaluation would also strengthen the project's reproducibility.

A potential commercial direction could combine the free, open-source local application with an optional hosted service for users without suitable hardware. Professional features, batch processing, and higher-quality reconstruction models could be explored later, subject to user validation and a review of the relevant model licences. These remain proposals rather than implemented capabilities.

Conclusion

2DorNot2D began with a simple question: could we make the journey from a hand-drawn idea to a usable 3D object less demanding? Within 24 hours, we developed a modular application that connects sketch preprocessing, diffusion-based image generation, 3D reconstruction, geometric evaluation, mesh export, and an interactive user interface.

Our experiments showed that selecting candidates according to their 3D silhouettes improved mean IoU from 0.880 with a single seed to 0.911 with the default best-of-three strategy across 12 real sketches.

Equally important, the project established a documented workflow, automated tests, CPU and GPU execution paths, and interfaces intended to support further development.

The system is not yet a replacement for professional 3D modelling. Its textures, unseen surfaces, fine structural details, and low-memory generation capabilities still need improvement. However, it demonstrates a practical starting point for making 3D creation more approachable.

For us, the value of 2DorNot2D lies not only in converting two-dimensional drawings into three-dimensional meshes, but also in turning an ambitious idea into a working, testable system under a demanding deadline. It is a foundation we can build on, improve through evidence, and eventually extend into a more capable tool for creators who want to spend less time translating their ideas into geometry and more time developing the ideas themselves.